Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Program: Software Engineering

Operating Systems

Laboratory Work No. 1

The OS as a Resource Manager

Track A: The Real OS, from the Outside (Linux)

Done by:
st. gr. FAF-241
Sitarciuc Dmitri

Checked by:
Reițman Patricia

Chișinău – 2026

Laboratory Work No. 1 – The OS as a Resource Manager

1. Introduction

The purpose of this laboratory work is to observe how an operating system manages the main resources of a computer: files, processes, memory and devices.

The work was performed in **GitHub Codespaces**, a cloud Linux environment that runs Ubuntu inside a Docker container on a Microsoft Azure virtual machine. I used it because a local Linux installation (WSL) could not be enabled on my Windows computer. Running inside a container changes some results compared with a normal virtual machine (for example, PID 1 and the list of disks), and these differences are explained in the observations.

2. System Information

The current user, kernel and uptime were checked with:

```
whoami
uname -a
uptime
```

Output:

```
codespace
Linux codespaces-84fcc1 6.8.0-1064-azure #72~22.04.1-Ubuntu SMP Wed Jul 22
23:39:45 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
 08:18:59 up 6 min,  0 user,  load average: 4.86, 5.13, 2.68
```

Figure 2.1 – Output of whoami, uname -a and uptime

The user is `codespace`, the default account that Codespaces creates in every environment. The kernel is a 64-bit Linux 6.8 build made for Azure, which shows that the container shares the kernel of the cloud virtual machine it runs on. The machine had been up for 6 minutes. The load average was high (about 5) because the environment had just started and was still launching its services. `0 user` appears because I was connected through the browser terminal, which is not registered as a normal login session.

3. Files and Directories

I inspected the main directories and then created, copied, renamed and deleted files in a demo folder:

```
pwd
ls -la /
ls -la ~
cd /etc && ls | head
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
chmod 600 note.txt
ls -l
rm renamed.txt
```

3.1. Observations

1. The files `note.txt` and `renamed.txt` are owned by user `codespace` and belong to group `codespace`.
2. After `chmod 600 note.txt`, `ls -l` shows `-rw-------`. The first `-` means a regular file. The next three characters `rw-` are the owner's rights (read and write, no execute). The next three `---` are the group's rights and the last three `---` are everyone else's: no access. Only the owner can now read or change the file.
3. Two directories under `/`: `/etc` holds system-wide configuration files (for example `adduser.conf`, `apt`, `bash.bashrc`), and `/home` holds the home directories of users (here `/home/codespace`).

3.2. Interesting Command Output

The same folder before and after `chmod 600`:

```
-rw-r--r-- 1 codespace codespace 24 Oct 23 08:18 note.txt
-rw-r--r-- 1 codespace codespace 24 Oct 23 08:18 renamed.txt

-rw------- 1 codespace codespace 24 Oct 23 08:18 note.txt
-rw-r--r-- 1 codespace codespace 24 Oct 23 08:18 renamed.txt
```

Figure 3.1 – ls -l of the demo folder before and after chmod 600 note.txt

Only the permissions of `note.txt` changed; its size, owner and contents stayed the same. The new files started as `-rw-r--r--` because the default umask in this environment is `0022`, which removes write permission for group and others. The root listing also showed a file `/.dockerenv`, which is a sign that the system runs inside a Docker container.

4. Processes

I listed the running processes, started a background `sleep` and looked at what the kernel stores about it in `/proc`:

```
ps aux | head
ps aux | wc -l
ps -p 1 -o pid,comm
sleep 300 & PID=$!
ps -ef | grep sleep
head -20 /proc/$PID/status
kill $PID
```

4.1. Observations

1. The process with PID 1 is `docker-init`, not `systemd`. In a normal Linux system PID 1 is `systemd`, the first process started by the kernel, which launches and supervises all services. Inside a container, PID 1 is the program that started the container. Here `docker-init` starts a shell that keeps the Codespace running, and every other process in the container descends from it.
2. `ps aux | wc -l` printed 29. One line is the header, so 28 processes were running. This is far fewer than on a full Ubuntu machine (usually 150–250) because the container only sees its own processes, not those of the host.
3. The status file of my `sleep` process shows `State: S (sleeping)`: the process exists but is waiting for its timer and does not use the CPU.

4.2. Interesting Command Output

```
Name:   sleep
State:  S (sleeping)
Pid:    2499
PPid:   2476
Uid:    1000  1000  1000  1000
VmPeak: 6116 kB
VmSize: 6116 kB
```

Figure 4.1 – Selected lines from /proc/2499/status of the test sleep process (excerpt of head -20)

`PPid 2476` is the bash process that ran my script, so the kernel keeps a parent–child tree of processes. `Uid 1000` is the numeric id of user `codespace`. The `ps -ef | grep sleep` output also showed other `sleep 1` processes started by the container's keep-alive loop, so even my test search found processes I did not start.

5. Memory Management

I checked the total and free memory, swap, and the real memory used by a single `sleep` process:

```
free -h
head -6 /proc/meminfo
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

5.1. Observations

1. The machine has 7.8 GiB of RAM (`MemTotal: 8131876 kB`). Only 184 MiB is completely free, but 5.3 GiB is available to programs, because most of the "used" space is file cache (5.4 GiB in `buff/cache`) that the OS can give back at any moment.
2. Swap is an area on disk that the OS uses as an extension of RAM: when memory runs out, rarely used pages are moved there. In this environment swap is `0B`, so none is configured.
3. The `sleep` process uses `VmRSS: 2052 kB`, about 2 MB of real RAM. Even a program that does nothing needs memory for its code, the C library and its stack. Its virtual size (`VmSize: 6116 kB`) is about three times larger, because the OS keeps in RAM only the pages the process actually touches.

5.2. Interesting Command Output

```
               total        used        free      shared  buff/cache
available
Mem:           7.8Gi       2.5Gi       184Mi        62Mi       5.4Gi
5.3Gi
Swap:             0B          0B          0B
```

Figure 5.1 – Output of free -h

The small `free` value looks alarming, but it is normal: Linux uses spare RAM as a file cache to speed up disk access instead of leaving it empty. The `available` column is the number that shows how much memory programs can really get.

6. Devices and Storage

I looked at disk usage, block devices, device files and mounted filesystems:

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

6.1. Observations

1. The root filesystem `/` is mounted on `overlay` (32 GB, 60% used), not on a disk partition. An overlay filesystem is how Docker builds a container's root: read-only image layers are stacked and a writable layer is placed on top. The `mount` output confirms this with a long list of `lowerdir=/var/lib/docker/overlay2/...` layers.
2. An example of a device interface is `/dev/sda`, a 30 GB virtual disk of the Azure machine. Its partition `/dev/sda1` holds the host system; parts of it are mounted into the container (for example `/vscode`). Another example is `/dev/sdc1`, a 45 GB disk mounted on `/tmp`.
3. "Everything is a file" means that in UNIX many resources, including disks, terminals and kernel information, are accessed through file paths with the same operations as normal files (open, read, write). For example, `/dev/sda` is a disk, `/proc/<pid>/status` is information about a process, and both are read like files.

6.2. Interesting Command Output

```
NAME    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop4     7:4    0    32G  0 loop /workspaces
sda       8:0    0    30G  0 disk
├─sda1    8:1    0  29.9G  0 part /vscode
├─sda14   8:14   0     4M  0 part
└─sda15   8:15   0   106M  0 part
sdb       8:16   0   512G  0 disk
└─sdb1    8:17   0   512G  0 part
sdc       8:32   0    45G  0 disk
└─sdc1    8:33   0    45G  0 part /tmp
```

Figure 6.1 – Output of lsblk (excerpt: read-only loop devices, sr0 and extra mount points of loop4 omitted)

The machine has three real disks (`sda` , `sdb` , `sdc`) and several `loop` devices, which are files used as virtual disks. My project folder `/workspaces` is stored on `loop4` . The `os-lab1` working folder itself takes only 16 KB.

7. Conclusion

The operating system manages four kinds of resources, and each can be observed from the terminal: files with `ls` and `chmod` , processes with `ps` and `/proc` , memory with `free` and `/proc/meminfo` , and storage with `df` , `lsblk` and `mount` .

Working in a container made the OS's role even clearer. The kernel is shared with the host machine, but the container sees only its own processes, starts from `docker-init` instead of `systemd` , and has a layered `overlay` root filesystem. This showed me that the OS decides not only how resources are used but also what each program is allowed to see.